

네트워크[7] - DNS & Domain

남궁명수

[DNS]

DNS(Domain Name System)는 사람이 읽기 쉬운 도메인 이름을 컴퓨터가 통신에 사용하는 IP 주소 등의 정보로 변환하는 분산형 계층 시스템이다.

사람이 사용하는 주소

www.example.com

DNS 조회

컴퓨터가 사용하는 주소

93.184.216.34

DNS의 핵심 역할은 다음과 같다.

도메인 이름에 대응하는 IP 주소나 기타 네트워크 정보를 찾아주는 것

DNS는 단순한 하나의 중앙 서버가 아니라 전 세계에 분산된 여러 DNS 서버가 계층적으로 협력하는 시스템이다.

[1] [DNS가 필요한 이유]

컴퓨터는 IP 주소를 이용해 목적지 호스트를 찾는다.

93.184.216.34

하지만 사람은 숫자로 된 IP 주소를 모두 기억하기 어렵다.

따라서 다음처럼 의미 있는 이름을 사용한다.

google.com

github.com

api.example.com

DNS가 도메인 이름을 IP 주소로 변환한다.

google.com

↓ DNS Lookup

142.250.xxx.xxx

↓ TCP 또는 UDP 통신

Google 서버 접속

중요한 점은 실제 IP 패킷이 도메인 이름을 목적지 주소로 사용하지 않는다는 것이다.

도메인 이름 → DNS로 IP 주소 조회
IP 주소 → 실제 IP 패킷의 목적지 주소

DNS 조회가 끝났다는 것은 주소를 찾았다는 뜻이지 웹페이지 데이터까지 받은 것은 아니다.

[2] [Domain]

Domain은 인터넷에서 특정 서비스나 조직을 식별하기 위해 사용하는 계층적인 이름이다.

예를 들어:

www	.	example	.	com	.
호스트		2단계 도메인		TLD	Root

DNS 계층은 오른쪽에서 왼쪽으로 해석한다.

. → com → example → www

[Root Domain]

DNS 계층의 최상위는 Root Domain이다. 표현상 마지막에 점을 붙인다.

www.example.com.
 ↑
 Root

평소에는 마지막 점을 생략한다.

www.example.com

Root DNS 서버는 모든 도메인의 실제 IP 주소를 직접 가지고 있는 것이 아니다.

대신 .com, .net, .org, .kr 같은 TLD를 담당하는 DNS 서버가 어디에 있는지 알려준다.

질문:

“www.example.com을 어디에서 찾을 수 있나요?”

Root:

“.com을 담당하는 TLD DNS 서버에 물어보세요.”

[TLD]

TLD(Top-Level Domain)는 Root 바로 아래의 최상위 도메인이다.

[일반 최상위 도메인]

```
.com  
.org  
.net  
.info
```

국가 코드 최상위 도메인

```
.kr → 대한민국  
.jp → 일본  
.uk → 영국
```

.com TLD 서버는 일반적으로 example.com의 최종 IP를 직접 알려주기보다, example.com을 담당하는 권한 있는 DNS 서버의 주소를 알려준다.

질문:

“www.example.com의 IP를 알고 있나요?”

.com TLD 서버:

“example.com을 담당하는 Authoritative DNS 서버에 물어보세요.”

[Second-Level Domain]

TLD 바로 왼쪽에 위치한 조직 또는 서비스의 주요 이름이다.

```
example.com  
google.com  
github.com
```

다음 예시에서:

```
example → Second-Level Domain  
com      → TLD
```

다만 국가별 도메인 구조에 따라 계층 표현이 다르게 보일 수 있다.

```
example.co.kr
```

일반적인 관점에서 등록 가능한 주요 이름이 example이고, co.kr이 조직 유형과 국가를 나타내는 계층으로 사용된다.

[Subdomain]

기본 도메인의 하위에 추가로 만든 영역이다.

```
api.example.com  
blog.example.com  
mail.example.com
```

여기서 다음 부분이 서브 도메인이다.

```
api  
blog  
mail
```

서브 도메인을 이용하면 서비스별 주소를 분리할 수 있다.

```
www.example.com → 웹 서비스  
api.example.com → API 서버  
mail.example.com → 메일 서버
```

[Hostname]

Hostname은 네트워크에서 특정 호스트나 서비스를 식별하는 이름이다.

```
www.example.com
```

www를 호스트 이름으로 설명하는 경우가 많다.

다만 실제 DNS에서는 www가 반드시 물리적인 컴퓨터 한 대를 의미하는 것은 아니다. 다음을 가리킬 수도 있다.

```
하나의 서버  
여러 서버의 IP 주소  
CDN  
로드 밸런서  
다른 도메인의 별칭
```

[FQDN]

FQDN(Fully Qualified Domain Name)은 DNS 계층에서 위치를 완전히 나타내는 도메인이다.

```
www.example.com.
```

마지막 루트 점까지 포함한 것이 엄밀한 FQDN 표현이다.

실무에서는 마지막 점을 생략한 www.example.com도 FQDN이라고 부르는 경우가 많다.

[3] [도메인과 URL의 차이]

도메인과 URL은 같은 것이 아니다.

예를 들어:

```
https://www.example.com:443/users/10?sort=name
```

각 부분은 다음과 같다.

https	→ Scheme 또는 Protocol
www.example.com	→ Domain
443	→ Port
/users/10	→ Path
sort=name	→ Query String

DNS가 처리하는 부분은 주로 도메인이다.

```
DNS 조회 대상: www.example.com
```

DNS는 다음 부분을 처리하지 않는다.

```
https
443
/users/10
sort=name
```

따라서 DNS 서버는 사용자가 /users/10을 요청했는지 알 필요가 없다. DNS는 도메인에 대응하는 IP 주소 등을 알려주는 역할만 한다.

[4] [DNS 레코드]

DNS 서버는 도메인에 관한 정보를 Resource Record 형태로 저장한다.

[A Record]

도메인 이름을 IPv4 주소에 연결한다.

```
example.com → 93.184.216.34
```

```
Type: A
```

```
Name: example.com
```

```
Value: 93.184.216.34
```

[AAAA Record]

도메인 이름을 IPv6 주소에 연결한다.

```
example.com → 2606:2800:220:1:248:1893:25c8:1946
```

IPv4 주소는 32비트지만 IPv6 주소는 128비트이므로 AAAA라는 이름을 사용한다.

[CNAME Record]

한 도메인 이름을 다른 도메인 이름의 별칭으로 연결한다.

```
www.example.com → example.com
```

또는:

```
cdn.example.com → provider.cdn.net
```

DNS Resolver는 CNAME이 가리키는 최종 도메인의 A 또는 AAAA 레코드를 추가로 조회해야 한다.

```
www.example.com
  ↓ CNAME
server.example.net
  ↓ A
203.0.113.10
```

[MX Record]

도메인의 이메일을 처리할 메일 서버를 지정한다.

```
example.com
→ mail.example.com
```

MX 레코드에는 우선순위 값이 포함될 수 있다.

```
10 mail1.example.com
20 mail2.example.com
```

일반적으로 값이 낮을수록 우선순위가 높다.

[NS Record]

해당 DNS 영역을 관리하는 권한 있는 네임서버를 지정한다.

```
example.com
→ ns1.example-dns.com
→ ns2.example-dns.com
```

[TXT Record]

임의의 텍스트 정보를 저장한다. 주로 다음 용도로 사용한다.

- 도메인 소유권 검증
- SPF
- DKIM
- 각종 서비스 인증

[SOA Record]

DNS Zone의 기본 관리 정보를 담는다. 대표적으로 다음 정보가 들어간다.

- Primary Name Server
- 관리자 정보
- Serial Number
- 갱신 주기
- 재시도 주기
- 만료 시간

[PTR Record]

IP 주소를 도메인 이름으로 찾는 역방향 DNS 조회에 사용한다.

도메인 → IP
A, AAAA 조회

IP → 도메인
PTR 조회

[SRV Record]

특정 서비스를 제공하는 서버의 호스트와 포트 정보를 지정한다.

[5] [Resolver]

Resolver는 애플리케이션의 DNS 질의를 처리하여 도메인에 대응하는 정보를 찾아주는 구성 요소다.

Resolver는 크게 두 가지로 구분할 수 있다.

Stub Resolver
Recursive Resolver

[Stub Resolver]

Stub Resolver는 사용자 컴퓨터 또는 운영체제에 있는 간단한 DNS 조회 기능이다.

브라우저나 curl 같은 애플리케이션이 도메인 조회를 요청하면 Stub Resolver가 설정된 DNS 서버에 질의를 전달한다.

애플리케이션
↓
OS Stub Resolver
↓
Recursive DNS Resolver

Linux에서는 애플리케이션이 일반적으로 다음과 같은 함수를 통해 이름 해석을 요청한다.

```
getaddrinfo();
```

예를 들어:

```
getaddrinfo("example.com", "443", ...);
```

애플리케이션 입장에서는 다음과 같이 요청하는 셈이다.

“example.com에 연결할 수 있는 IP 주소 정보를 알려주세요.”

Stub Resolver는 일반적으로 DNS 계층 전체를 직접 조회하지 않고 Recursive Resolver에 작업을 맡긴다.

[Recursive Resolver]

Recursive Resolver는 클라이언트를 대신하여 DNS 계층을 따라가며 최종 답을 찾는다.

다음과 같은 DNS 서버가 Recursive Resolver 역할을 할 수 있다.

- 인터넷 서비스 제공업체의 DNS 서버
- 회사 또는 학교의 DNS 서버
- 공유기나 내부 DNS 서버
- Google Public DNS
- Cloudflare DNS
- 통신사의 DNS Resolver

대표적인 공개 Resolver 주소 예시는 다음과 같다.

- Google Public DNS
8.8.8.8
- Cloudflare DNS
1.1.1.1

Recursive Resolver는 다음 역할을 수행한다.

- 클라이언트의 DNS 질의 수신
- 캐시 확인
- Root DNS Server 질의
- TLD DNS Server 질의
- Authoritative DNS Server 질의
- 최종 결과 캐싱
- 클라이언트에게 결과 반환

[6] [DNS 서버의 계층 구조]

DNS Lookup에는 주로 다음 서버들이 참여한다.

Recursive Resolver
Root DNS Server
TLD DNS Server
Authoritative DNS Server

[Root DNS Server]

Root DNS Server는 DNS 계층의 최상위에 있다.

하지만 일반적으로 `www.example.com`의 실제 IP 주소를 직접 알려주는 역할을 하는 것은 아니다.
대신 다음과 같이 안내한다.

`.com`에 대한 정보는 `.com` TLD 네임서버에 물어보세요.

질문:

`www.example.com`의 IP는?

Root 응답:

정확한 IP는 모르지만 `.com` TLD 서버는 알고 있습니다.

[TLD DNS Server]

TLD 서버는 `.com`, `.org`, `.kr` 같은 특정 최상위 도메인 영역을 담당한다.

`.com` TLD 서버는 다음과 같이 안내한다.

`example.com`에 대한 정보는 해당 권한 있는 네임서버에 물어보세요.

질문:

`www.example.com`의 IP는?

`.com` TLD 응답:

`example.com`의 Authoritative Server는 이쪽입니다.

[Authoritative DNS Server]

Authoritative DNS Server는 특정 도메인에 대한 실제 DNS 레코드를 관리하는 권한 있는 서버다.

`example.com` Zone
├ A Record
├ AAAA Record
├ CNAME Record
├ MX Record
└ TXT Record

www.example.com의 A 레코드를 가지고 있다면 최종 IP 주소를 반환한다.

www.example.com → 93.184.216.34

[7] [DNS Lookup 전체 과정]

사용자가 브라우저에 다음 주소를 입력했다고 하자.

https://www.example.com

전체 과정은 대략 다음과 같다.

```
브라우저
↓
브라우저 DNS 캐시
↓
운영체제 DNS 캐시 및 hosts 파일
↓
Recursive Resolver
↓
Root DNS Server
↓
.com TLD DNS Server
↓
example.com Authoritative DNS Server
↓
IP 주소 반환
```

[1단계: 브라우저 캐시 확인]

브라우저는 이전에 조회한 DNS 결과를 자체적으로 캐싱할 수 있다.

브라우저 DNS 캐시에 결과 있음
→ 즉시 IP 주소 사용

결과가 없다면 운영체제에 이름 해석을 요청한다.

[2단계: 운영체제 캐시와 hosts 파일 확인]

운영체제는 DNS 캐시와 로컬 이름 설정을 확인한다.

Linux나 macOS의 hosts 파일은 일반적으로 다음 위치에 있다.

/etc/hosts

예를 들어:

```
127.0.0.1 localhost
192.168.0.10 dev.example.com
```

dev.example.com을 조회하면 DNS 서버에 묻지 않고 설정에 따라 192.168.0.10을 사용할 수 있다. 실제 이름 해석 순서는 운영체제 설정에 따라 달라질 수 있다.

Linux에서는 /etc/nsswitch.conf의 hosts 항목을 통해 이름 해석 순서를 설정할 수 있다.

```
hosts: files dns
```

이는 일반적으로 로컬 파일을 먼저 확인하고 이후 DNS를 사용한다는 의미다.

[3단계: Recursive Resolver에 질의]

로컬에서 결과를 찾지 못하면 Stub Resolver는 설정된 Recursive Resolver에 질의를 보낸다.

```
클라이언트:
www.example.com의 A 레코드를 알려주세요.
```

Recursive Resolver는 먼저 자신의 캐시를 확인한다.

```
캐시에 결과 있음
→ 즉시 클라이언트에게 반환

캐시에 결과 없음
→ DNS 계층 조회 시작
```

클라이언트는 일반적으로 최종 결과를 요구하는 재귀 질의를 Resolver에 전달한다.

[4단계: Root DNS Server에 질의]

Recursive Resolver가 Root DNS Server에 묻는다.

```
Resolver:
www.example.com의 A 레코드를 알려주세요.
```

Root 서버는 일반적으로 실제 A 레코드를 알려주지 않고 .com TLD 네임서버 정보를 반환한다.

```
Root:
.com 영역은 다음 TLD 네임서버에 물어보세요.
```

이를 Referral, 즉 다른 DNS 서버로의 안내라고 한다.

[5단계: TLD DNS Server에 질의]

Recursive Resolver는 안내받은 .com TLD 서버에 질의한다.

Resolver:

www.example.com의 A 레코드를 알려주세요.

.com TLD 서버는 example.com을 관리하는 Authoritative DNS Server 정보를 반환한다.

.com TLD:

example.com은 다음 Authoritative Server에 물어보세요.

[6단계: Authoritative DNS Server에 질의]

Recursive Resolver는 example.com의 Authoritative DNS Server에 질의한다.

Resolver:

www.example.com의 A 레코드를 알려주세요.

권한 있는 서버는 자신이 관리하는 DNS 레코드에서 결과를 찾아 반환한다.

Authoritative Server:

www.example.com의 A 레코드는 93.184.216.34입니다.

CNAME이 설정되어 있다면 다른 도메인 이름을 반환할 수 있다.

www.example.com

→ CNAME server.cdn-provider.net

이 경우 Resolver는 server.cdn-provider.net의 A 또는 AAAA 레코드를 추가로 조회한다.

[7단계: Resolver가 결과를 캐싱]

Recursive Resolver는 최종 결과를 일정 시간 캐시에 저장한다.

www.example.com

→ 93.184.216.34

→ TTL 동안 캐싱

이후 다른 클라이언트가 같은 정보를 요청하면 Root부터 다시 조회하지 않고 캐시에서 바로 응답할 수 있다.

[8단계: 클라이언트에게 IP 반환]

Recursive Resolver가 클라이언트의 Stub Resolver에게 최종 결과를 반환한다.

www.example.com → 93.184.216.34

운영체제는 결과를 애플리케이션에 전달한다.

[9단계: 서버와 네트워크 연결]

DNS Lookup은 IP 주소를 찾는 과정일 뿐이다.
IP 주소를 얻은 뒤에야 실제 서버 연결이 시작된다.
HTTPS라면:

DNS Lookup

↓

서버 IP 확인

↓

TCP 3-Way Handshake

↓

TLS Handshake

↓

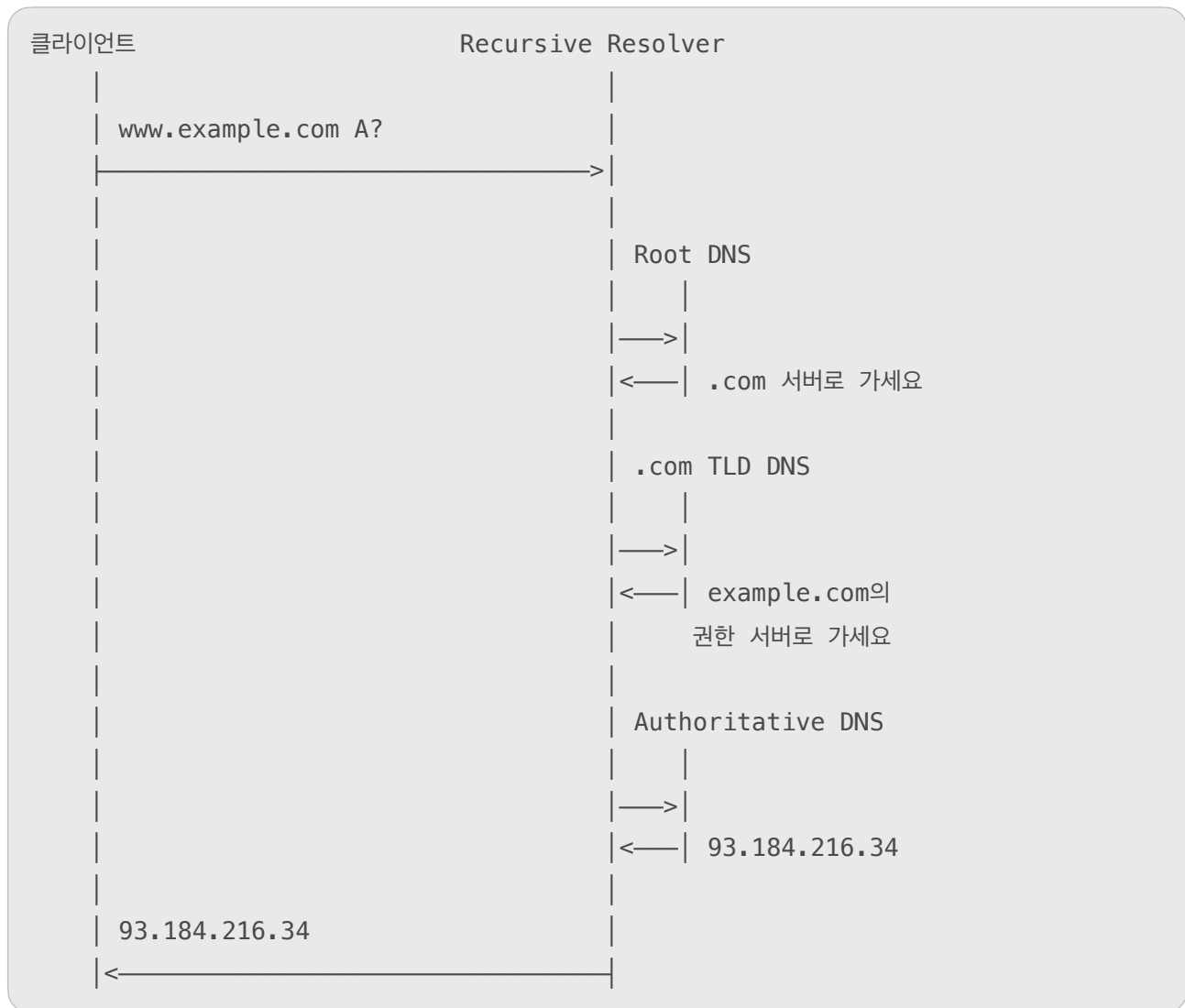
암호화된 HTTP 요청

↓

HTTP 응답

즉, DNS가 완료됐다고 해서 웹페이지 데이터까지 받은 것은 아니다.

[8] [DNS Lookup 그림]



핵심 흐름은 다음과 같다.

클라이언트
→ Recursive Resolver
→ Root
→ TLD
→ Authoritative
→ Recursive Resolver
→ 클라이언트

[9] [재귀적 질의와 반복적 질의]

DNS Lookup에서는 재귀적 질의와 반복적 질의를 구분해야 한다.

[재귀적 질의]

질의를 받은 DNS 서버가 최종 결과를 찾아서 반환해야 하는 방식이다.

클라이언트는 Recursive Resolver에 다음과 같이 요청한다.

제가 직접 다른 서버에 묻지 않도록 최종 IP 주소까지 찾아서 알려주세요.

클라이언트 → Recursive Resolver

클라이언트는 일반적으로 하나의 DNS 서버에 질의하고 최종 결과를 받는다.

[반복적 질의]

DNS 서버가 자신이 최종 답을 모르면 다음에 질의할 서버 정보를 알려주는 방식이다.

Resolver → Root:

www.example.com은?

Root:

.com 서버에 물어보세요.

Resolver → .com TLD:

www.example.com은?

TLD:

example.com 권한 서버에 물어보세요.

Recursive Resolver가 Root, TLD, Authoritative 서버를 따라가는 과정은 반복적 질의 형태로 설명할 수 있다.
정리하면:

클라이언트 → Recursive Resolver

재귀적 질의:

“최종 답까지 찾아주세요.”

Recursive Resolver → DNS 계층 서버들

반복적 질의:

“모르면 다음 서버를 알려주세요.”

[10] [DNS Cache와 TTL]

DNS 조회를 매번 Root부터 다시 수행하면 다음 문제가 발생한다.

- 응답 지연 증가
- DNS 서버 부하 증가
- 네트워크 트래픽 증가

따라서 DNS는 조회 결과를 캐싱한다.

[TTL]

TTL(Time To Live)은 DNS 레코드를 캐시에 얼마나 오래 저장할 수 있는지를 나타낸다.

A Record:

example.com → 93.184.216.34

TTL: 300초

Resolver는 최대 300초 동안 해당 결과를 재사용할 수 있다.

첫 번째 요청:

Root → TLD → Authoritative 조회

300초 이내 다음 요청:

캐시에서 바로 응답

TTL이 만료되면 Resolver는 필요할 때 다시 권한 있는 서버에 질의해 최신 정보를 가져온다.

[TTL이 긴 경우]

장점:

- DNS 조회 횟수 감소
- 응답 속도 향상
- 권한 서버 부하 감소

단점:

- IP 주소를 변경해도 이전 값이 오래 남을 수 있음

[TTL이 짧은 경우]

장점:

- DNS 변경 사항이 비교적 빠르게 반영됨
- 장애 조치나 서버 변경에 유리

단점:

- DNS 조회 횟수 증가
 - Resolver와 권한 서버 부하 증가 가능
-

[11] [DNS 변경이 즉시 적용되지 않는 이유]

도메인의 IP 주소를 변경했는데도 일부 사용자에게 이전 서버가 보일 수 있다.

기존 레코드:

example.com → 192.0.2.10

변경된 레코드:

example.com → 192.0.2.20

Recursive Resolver, 운영체제, 브라우저 등에 기존 결과가 TTL 동안 남아 있을 수 있기 때문이다.

이를 흔히 DNS 전파라고 표현하지만, 실제로는 변경 내용이 전 세계에 순차적으로 복사되는 것뿐 아니라 **각 캐시의 기존 TTL이 만료되는 과정**이 중요한 원인이다.

[12] [Negative Caching]

DNS는 존재하는 레코드뿐 아니라 존재하지 않는다는 응답도 캐싱할 수 있다.

질의:

wrong.example.com이 있나요?

응답:

NXDOMAIN

NXDOMAIN은 해당 도메인 이름이 존재하지 않는다는 의미다.

Resolver는 이 결과도 일정 시간 캐싱할 수 있다.

따라서 잘못된 도메인을 먼저 조회한 후 바로 DNS 레코드를 추가해도 잠시 동안 계속 존재하지 않는다는 결과가 반환될 수 있다.

[13] [DNS는 UDP인가, TCP 인가?]

DNS는 전통적으로 UDP 53번 포트를 주로 사용한다.

일반적인 DNS 질의·응답

→ UDP 53

UDP를 사용하는 이유는 다음과 같다.

- 연결 설정이 필요 없음
- 일반적인 질의와 응답 크기가 작음
- 빠르게 요청과 응답 가능

하지만 DNS가 항상 UDP만 사용하는 것은 아니다. 다음 상황에서는 TCP 53번 포트를 사용할 수 있다.

- UDP 응답이 잘려서 TC 플래그가 설정된 경우
- 큰 DNS 응답
- DNSSEC으로 응답 크기가 커진 경우
- DNS Zone Transfer
- 클라이언트와 서버가 TCP 사용을 선택한 경우

일반 질의:

UDP 사용

UDP 응답이 너무 큼:

TCP로 다시 질의 가능

DNS Zone Transfer에는 대표적으로 다음이 있다.

AXFR → 전체 Zone 전송

IXFR → 변경된 Zone 정보 전송

[14] [DNS 암호화]

기존의 일반 DNS 질의는 암호화되지 않으므로 네트워크에서 도메인 조회 내용을 관찰하거나 변조할 가능성이 있다. 이를 보완하는 기술로 다음 방식이 있다.

[DoT]

DNS over TLS는 DNS 메시지를 TLS로 암호화한다.

DNS

↓

TLS

↓

TCP

일반적으로 853번 포트를 사용한다.

[DoH]

DNS over HTTPS는 DNS 질의를 HTTPS로 전송한다.

DNS

↓

HTTPS

↓

TLS

일반적인 HTTPS와 같은 443번 포트를 사용한다.

[DoQ]

DNS over QUIC은 DNS 메시지를 QUIC을 통해 전송한다.

[14] [DNSSEC]

DNSSEC은 DNS 응답에 전자서명을 추가하여 응답의 출처와 무결성을 검증할 수 있도록 한다.

DNSSEC이 제공하는 주요 기능은 다음과 같다.

- DNS 응답이 권한 있는 출처에서 왔는지 확인
- DNS 응답이 전송 중 변조되지 않았는지 확인

하지만 DNSSEC 자체가 DNS 질의 내용을 암호화하는 것은 아니다.

DNSSEC

- 응답의 진위와 무결성 검증
- 질의 내용 암호화는 아님

DoH·DoT

- DNS 통신 구간 암호화

[16] [하나의 도메인이 여러 IP를 가질 수 있는가?]

하나의 도메인에 여러 A 또는 AAAA 레코드를 설정할 수 있다.

example.com → 192.0.2.10

example.com → 192.0.2.11

example.com → 192.0.2.12

이 구조는 다음 목적으로 사용할 수 있다.

- 부하 분산
- 서버 장애 대응
- 지역별 서버 선택
- CDN
- 지연 시간이 짧은 서버 제공

따라서 도메인 하나가 항상 고정된 IP 하나만 의미하는 것은 아니다.

같은 도메인을 조회해도 다음 조건에 따라 다른 IP를 받을 수 있다.

- 조회 시점
 - 사용자 위치
 - DNS Resolver 위치
 - 서버 상태
 - CDN 정책
 - DNS 캐시 상태
-

[17] [DNS와 CDN]

CDN(Content Delivery Network)

전 세계 곳곳에 분산된 서버 네트워크를 통해 웹 콘텐츠를 사용자에게 가장 빠르고 안정적으로 전달하는 기술이다.

CDN은 DNS를 이용해 사용자와 가까운 서버로 연결할 수 있다.

한국 사용자

↓ DNS 조회

한국 또는 인접 지역 CDN 서버 IP 반환

미국 사용자

↓ DNS 조회

미국 지역 CDN 서버 IP 반환

사용자는 같은 도메인으로 접속하지만 DNS 응답은 위치나 네트워크 상태에 따라 달라질 수 있다.

www.example.com

한국 → 203.0.113.10

미국 → 198.51.100.20

[18] [Resolver와 Authoritative Server의 차이]

구분	Recursive Resolver	Authoritative DNS Server
주요 역할	클라이언트를 대신해 답을 찾음	자신이 관리하는 도메인의 공식 답 제공
캐시	조회 결과 캐싱	원본 Zone 정보 관리
질의 대상	Root, TLD, 권한 서버	Resolver의 질의를 받음
최종 사용자 요청	주로 직접 받음	일반적으로 Resolver를 통해 받음
예시	ISP DNS, 8.8.8.8, 1.1.1.1	특정 도메인의 네임서버

쉽게 말하면:

Recursive Resolver

→ 여러 기관에 물어보고 답을 찾아주는 검색 담당자

Authoritative DNS Server

→ 해당 도메인의 공식 정보를 가진 담당 기관

[19] [dig로 DNS 조회하기]

도메인의 A 레코드를 조회하려면 다음 명령을 사용할 수 있다.

```
dig example.com
```

간단한 IP 결과만 확인하려면:

```
dig +short example.com
```

특정 레코드를 조회하려면:

```
dig example.com A
dig example.com AAAA
dig example.com MX
dig example.com NS
dig example.com TXT
```

특정 DNS Resolver에 직접 질의하려면:

```
dig @8.8.8.8 example.com
```

DNS 계층을 따라가는 과정을 확인하려면:

```
dig +trace example.com
```

+trace를 사용하면 개념적으로 다음 흐름을 확인할 수 있다.

```
Root
↓
TLD
↓
Authoritative DNS Server
↓
최종 레코드
```

macOS에서는 기본 설치 상태에 따라 dig를 사용할 수 있다. 간단하게 이름 해석 결과를 확인하려면 다음 명령도 사용할 수 있다.

```
nslookup example.com
```

[20] [프로그램에서 DNS Lookup]

[c언어]

C 프로그램에서는 일반적으로 `getaddrinfo()`를 사용한다.

```
struct addrinfo *result;

getaddrinfo(
    "example.com",
    "443",
    NULL,
    &result
);
```

`getaddrinfo()`는 IPv4와 IPv6를 포함한 연결 가능한 주소 정보를 반환할 수 있다.

[Golang]

Go에서는 다음처럼 DNS 조회를 할 수 있다.

```
ips, err := net.LookupHost("example.com")
if err != nil {
    panic(err)
}

for _, ip := range ips {
    fmt.Println(ip)
}
```

또는 서버에 연결할 때:

```
conn, err := net.Dial("tcp", "example.com:443")
```

`net.Dial()`은 도메인이 입력되면 내부적으로 이름 해석을 수행한 뒤 얻은 IP 주소를 이용해 연결을 시도한다. 개념적인 흐름은 다음과 같다.

```
net.Dial("tcp", "example.com:443")
    ↓
DNS Lookup
    ↓
IP 주소 획득
    ↓
socket()
    ↓
connect()
    ↓
TCP 3-Way Handshake
```

[21] [전체 웹 접속 과정]

사용자가 브라우저에 다음 주소를 입력했다고 하자.

```
https://www.example.com/users
```

전체 과정은 다음과 같다.

1. URL 분석
Scheme: https
Domain: www.example.com
Path: /users
2. DNS Lookup
www.example.com → 서버 IP 확인
3. 라우팅과 로컬 전송 준비
라우팅 테이블 확인
게이트웨이 MAC 주소 확인
4. TCP 연결
서버 443번 포트와 3-Way Handshake
5. TLS Handshake
인증서 검증 및 대칭키 생성
6. HTTP 요청
GET /users
7. HTTP 응답
200 OK + Response Body
8. 연결 재사용 또는 종료
Keep-Alive 또는 연결 종료